第 9 章練習解答

CH09-E1

1.

$$(2-i)(3+2i) = 6 + 4i - 3i - 2i^2 = 8 + i.$$

$$\frac{2-i}{3+2i} = \frac{(2-i)(3-2i)}{13} = \frac{4-7i}{13}.$$

2.

$$|1 + i\sqrt{3}| = 2, \qquad 1 + i\sqrt{3} = 2e^{i\pi/3}.$$

3.

$$(1 + i\sqrt{3})^6 = 2^6 e^{i6\pi/3} = 64 e^{2\pi i} = 64.$$

4.

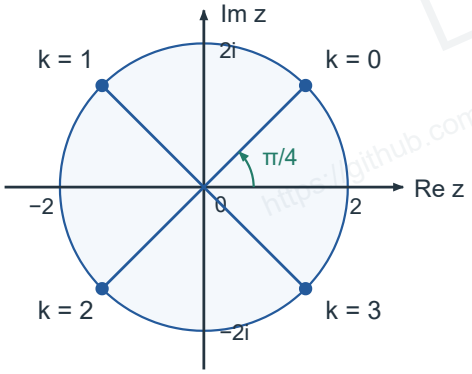
$$z^4 = -16 = 16e^{i\pi}.$$

四個根為

$$z_k = 2e^{i(\pi + 2\pi k)/4}, \quad k = 0, 1, 2, 3.$$

幅角為 $\pi/4, 3\pi/4, 5\pi/4, 7\pi/4$，四點等距，模皆為 2。

$z^4 = -16$：半徑 2，幅角相隔 π/2



角度由正實軸逆時針量起

k	$\theta_k = (\pi + 2\pi k)/4$	z_k
0	π/4	√2 + i√2
1	3π/4	−√2 + i√2
2	5π/4	−√2 − i√2
3	7π/4	√2 − i√2

$|z_k| = 2$；$4\theta_k = \pi + 2\pi k$，所以 $z_k^4 = -16$

CH09-E2

1.

$$\|u\|^2 = |1+i|^2 + |2-i|^2 = 7,$$

$$\|v\|^2 = 7.$$

$$u^H v = (1-i)(2-i) + (2+i)(1+i).$$

第一項是 $1 - 3i$，第二項是 $1 + 3i$，所以

$$u^H v = 2.$$

$$v^H u = 2.$$

第 9 章練習解答 （續）

2.

$$u^H v = \overline{v^H u} = \overline{2} = 2.$$

3. 本題用 $\langle u, v \rangle = u^H v$，共軛作用在第一個引數；因 $u^H v = 2 \neq 0$，所以不正交。若改用共軛作用在第二個引數的有效內積慣例，內積值會取共軛，是否為零不變，因此正交性不因這兩種慣例而改變。不能把沒有共軛的 $u^T v$ 當成複 Euclidean inner product。

4.

$$q = \frac{1}{\sqrt{7}}(1+i, 2-i)^T,$$

$$q^H q = \frac{1}{7} u^H u = 1.$$

CH09-E3

1.

$$S_{12} = i, \qquad S_{21} = -i = \overline{i},$$

對角元素 $2, 2$ 都是實數，所以 $S^H = S$。

2.

$$p_S(\lambda) = (2-\lambda)^2 - 1 = (\lambda - 1)(\lambda - 3).$$

特徵值 $1, 3$ 都是實數。

3. 對 $\lambda = 3$：

$$-z_1 + i z_2 = 0 \Rightarrow z_1 = i z_2.$$

$$q_1 = \frac{1}{\sqrt{2}}(i, 1)^T.$$

對 $\lambda = 1$：

$$z_1 + i z_2 = 0 \Rightarrow z_1 = -i z_2.$$

$$q_2 = \frac{1}{\sqrt{2}}(-i, 1)^T.$$

4.

$$Q = \frac{1}{\sqrt{2}} \begin{bmatrix} i & -i \\ 1 & 1 \end{bmatrix}.$$

$$Q^H Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

因此

$$S = Q \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} Q^H.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答 （續）

CH09-E4

1.

$$(A^H)^H = A.$$

對 product：

$$(AB)^H_{ij} = \overline{\sum_k A_{jk}B_{ki}} = \sum_k \overline{B_{ki}A_{jk}} = (B^H A^H)_{ij}.$$

2. $A^H = -A$，令 $Au = \lambda u$，左乘 u^H：

$$\lambda u^H u = u^H Au = -u^H A^H u = -(Au)^H u = -\overline{\lambda} u^H u.$$

故 $\lambda = -\overline{\lambda}$，eigenvalue 是 pure imaginary 或 zero。

3.

$$1 = \det(Q^H Q) = |\det Q|^2,$$

所以 $|\det Q| = 1$。

4. Hermitian 正定矩陣滿足

$$z^H Az = \lambda z^H z > 0$$

這對非零特徵向量成立，故 $\lambda > 0$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH09-E5

1. 本題矩陣為

$$Q = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}.$$

$$Q^T Q = \frac{1}{2}\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = I.$$

2. $u = (1,0)^T, v = (0,1)^T$，且 $u^H v = 0$。Unitary 矩陣保持內積：

$$(Qu)^H (Qv) = u^H Q^H Qv = 0.$$

長度亦保持 1。

3.

$$\det(Q - \lambda I) = \lambda^2 - \sqrt{2}\lambda + 1.$$

$$\lambda = e^{\pm i\pi/4}, \qquad |\lambda| = 1.$$

4. Q 是實矩陣，但 $Q^T = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \neq Q$，所以不是 Hermitian；其非實特徵值也排除 Hermitian。Unitary 與 Hermitian 可以重疊，例如 I，但一般 unitary 矩陣不必是 Hermitian。

第 9 章練習解答 （續）

CH09-E6

1. $\omega = i$：

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}.$$

2. $c = (1, 2, 0, -1)^T$：

$$y_0 = 2,$$
$$y_1 = 1 + 2i + 0 + i = 1 + 3i,$$
$$y_2 = 1 - 2 + 0 + 1 = 0,$$
$$y_3 = 1 - 2i + 0 - i = 1 - 3i.$$

所以

$$y = (2, 1 + 3i, 0, 1 - 3i)^T.$$

3.

$$F_4^H = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}.$$
$$F_4^H y = (4, 8, 0, -4)^T,$$

除以 4 得 $c = (1, 2, 0, -1)^T$。

4.

$$\|c\|^2 = 1 + 4 + 0 + 1 = 6.$$
$$\|y\|^2 = 4 + 10 + 0 + 10 = 24 = 4\|c\|^2.$$

CH09-E7

1. 偶數索引係數為

$$c' = (c_0, c_2, c_4, c_6)^T,$$

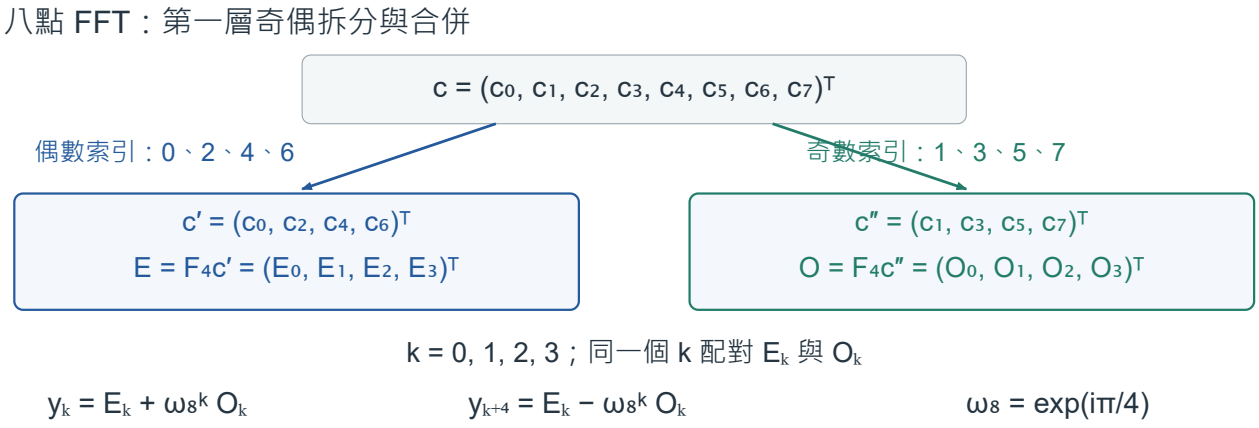
奇數索引係數為

$$c'' = (c_1, c_3, c_5, c_7)^T.$$

對兩者分別做大小為 4 的 DFT，得到 $E = F_4 c'$、$O = F_4 c''$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答 （續）



2. $j = 0, \dots, 3$：

$$y_j = E_j + \omega_8^j O_j,$$
$$y_{j+4} = E_j - \omega_8^j O_j.$$

twiddle 因數是 ω_8^j。

3.

$$T(8) = 2T(4) + 8,$$
$$T(4) = 2T(2) + 4, \qquad T(2) = 2T(1) + 2.$$

每層做線性工作量，共 $\log_2 n$ 層，故

$$T(n) = \Theta(n \log n).$$

4. Radix-2 要求 n 可分解為 2 的冪。其他 n 可用混合基底、Bluestein 或補零；補零會改變訊號長度與頻率格點。

CH09-E8

1. 令 q_j 為 Q 的第 j 個 column。$Q^H Q = I$ 給

$$q_i^H q_j = \delta_{ij},$$

所以這 n 個 columns 是 $\mathbb{C}^n$ 的正交歸一基底。對方陣，$Q^H Q = I$ 同時給出 $Q^{-1} = Q^H$，因此 $QQ^H = I$。若將第 i 個 row 轉寫成 column $r_i = (Q_{i1}, \dots, Q_{in})^T$，則

$$r_i^H r_j = \overline{(QQ^H)_{ij}} = \delta_{ij}.$$

故 rows 也構成正交歸一基底；必須區分兩個矩陣乘積各自直接檢查哪一組向量。

2. Hermitian 矩陣 $S = S^H$ 的對角元素是實數，因此跡是實數；行列式為實特徵值的乘積，也是實數。

3.

$$(cF_n)^H (cF_n) = |c|^2 nI.$$

第 9 章練習解答 （續）

Unitary 要求

$$|c| = \frac{1}{\sqrt{n}}.$$

> 任意全域相位 $e^{i\phi}$ 都不會改變範數。

4. 使用 ω 或 ϖ 只交換正、逆轉換的標記，或對應到取共軛與反向排列。正規化後仍得到

$$F_n F_n^H = nI.$$

因此 Parseval 公式中的 n 因數由正規化方式決定，不由符號慣例決定。

CH09-E9

1. Bit reversal 把 $0, 1, 2, 3$ 排成 $0, 2, 1, 3$，所以

$$(c_0, c_2, c_1, c_3) = (1, 0, 2, -1).$$

2. 第一層：

$$(1, 0) \mapsto (1, 1), \qquad (2, -1) \mapsto (1, 3).$$

第二層：

$$(1, 1) \mapsto (2, 0), \qquad (1, 3) \mapsto (1 + 3i, 1 - 3i).$$

3. 因此

$$F_4 c = (2, 1 + 3i, 0, 1 - 3i)^T.$$

4. 直接計算

$$y_1 = 1 + 2i + 0(-1) + (-1)(-i) = 1 + 3i,$$
$$y_3 = 1 - 2i + 0(-1) + (-1)i = 1 - 3i,$$

與 FFT 中間層一致。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答 （續）

原創驗算程式附錄: complex_fft.py

```
"""Verify Hermitian, unitary, DFT and radix-2 FFT examples."""

from cmath import exp, isclose, pi

Complex = complex
Vec = tuple[Complex, ...]
Mat = tuple[tuple[Complex, ...], ...]


def conjugate_transpose(a: Mat) -> Mat:
    return tuple(tuple(a[j][i].conjugate() for j in range(len(a))) for i in range(len(a[0])))


def matvec(a: Mat, x: Vec) -> Vec:
    return tuple(sum(a[i][j] * x[j] for j in range(len(x))) for i in range(len(a)))


def matmul(a: Mat, b: Mat) -> Mat:
    return tuple(
        tuple(sum(a[i][k] * b[k][j] for k in range(len(b))) for j in range(len(b[0])))
        for i in range(len(a))
    )


def matrix_power(a: Mat, exponent: int) -> Mat:
    result = tuple(tuple(1.0 + 0j if i == j else 0j for j in range(len(a))) for i in
range(len(a)))
    while exponent:
        if exponent & 1:
            result = matmul(result, a)
        a = matmul(a, a)
        exponent >>= 1
    return result


a = ((2 + 0j, 1j), (-1j, 3 + 0j))
assert conjugate_transpose(a) == a
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答 （續）

```
traces = sum(a[i][i] for i in range(2))
assert abs(traces.imag) < 1e-12

q = (
    (1 / 2**0.5, 1 / 2**0.5),
    (-1 / 2**0.5, 1 / 2**0.5),
)
qh = conjugate_transpose(q)
qhq = matmul(qh, q)
assert qh != q
assert all(
    abs(qhq[i][j] - (1 if i == j else 0)) < 1e-12
    for i in range(2)
    for j in range(2)
)


def naive_dft(values: Vec) -> Vec:
    n = len(values)
    omega = exp(2j * pi / n)
    return tuple(
        sum((omega ** (j * k)) * values[k] for k in range(n))
        for j in range(n)
    )


def fft(values: Vec) -> Vec:
    n = len(values)
    if n == 1:
        return values
    if n & (n - 1):
        raise ValueError("power-of-two length required")
    even = fft(values[0::2])
    odd = fft(values[1::2])
    omega = exp(2j * pi / n)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答 （續）

```
    output = [0j] * n
    for j in range(n // 2):
        twiddle = (omega**j) * odd[j]
        output[j] = even[j] + twiddle
        output[j + n // 2] = even[j] - twiddle
    return tuple(output)


samples = (1 + 0j, 2 + 0j, 0 + 0j, -1 + 0j, 3 + 0j, -2 + 0j, 1 + 0j, 0.5 + 0j)
naive = naive_dft(samples)
fast = fft(samples)
assert all(isclose(naive[i], fast[i], abs_tol=1e-12) for i in range(8))

s4 = samples[:4]
y4 = naive_dft(s4)
assert abs(sum(abs(value) ** 2 for value in y4) - 4 * sum(abs(value) ** 2 for value in s4)) <
1e-10

signal = (1.0 + 0j, 2.0 + 0j, 0.0 + 0j, -1.0 + 0j)
reversed_signal = (signal[0], signal[2], signal[1], signal[3])
assert reversed_signal == (1.0 + 0j, 0.0 + 0j, 2.0 + 0j, -1.0 + 0j)
stage1 = (
    reversed_signal[0] + reversed_signal[1],
    reversed_signal[0] - reversed_signal[1],
    reversed_signal[2] + reversed_signal[3],
    reversed_signal[2] - reversed_signal[3],
)
assert stage1 == (1.0 + 0j, 1.0 + 0j, 1.0 + 0j, 3.0 + 0j)
stage2 = (
    stage1[0] + stage1[2],
    stage1[1] + 1j * stage1[3],
    stage1[0] - stage1[2],
    stage1[1] - 1j * stage1[3],
)
expected_trace = (2.0 + 0j, 1.0 + 3j, 0.0 + 0j, 1.0 - 3j)
assert all(isclose(stage2[i], expected_trace[i], abs_tol=1e-12) for i in range(4))
assert all(isclose(naive_dft(signal)[i], expected_trace[i], abs_tol=1e-12) for i in range(4))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答 （續）

```
def fft4_trace(signal):
    """Positive-sign DFT: bit reversal, adjacent pairs, then pairs (0,2)/(1,3)."""
    if len(signal) != 4:
        raise ValueError("exactly four coefficients required")
    reordered = (signal[0], signal[2], signal[1], signal[3])
    first = (
        reordered[0] + reordered[1], reordered[0] - reordered[1],
        reordered[2] + reordered[3], reordered[2] - reordered[3],
    )
    second = (
        first[0] + first[2], first[1] + 1j * first[3],
        first[0] - first[2], first[1] - 1j * first[3],
    )
    return reordered, first, second


counterexample = (2, 5, 7, -1)
reordered, first, second = fft4_trace(counterexample)
assert reordered == (2, 7, 5, -1) and first == (9, -5, 4, 6)
assert second == (13, -5 + 6j, 5, -5 - 6j)
assert all(isclose(a, b, abs_tol=1e-12) for a, b in zip(second, naive_dft(counterexample)))
from itertools import product
for coefficients in product(range(-2, 3), repeat=4):
    traced = fft4_trace(coefficients)[2]
    assert all(isclose(a, b, abs_tol=1e-12) for a, b in zip(traced, naive_dft(coefficients)))

print("Hermitian matrix trace imaginary part =", traces.imag)
print("Q^H Q =", qhq)
print("DFT and FFT match:", all(isclose(naive[i], fast[i], abs_tol=1e-12) for i in range(8)))
print("Parseval check passed")
print("n=4 FFT stages =", stage1, stage2)
```

參考資料與署名

Gilbert Strang. Introduction to Linear Algebra, 5th Edition, 2016.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study